שלום **איתמר כהן** (יציאה)

מערכות הפעלה - 20594

מדעי המחשב

סמסטר 2021ב: מבחן בית - הערכה חלופית

מרכזי הוראה: דוד שריאל, יקיר צדוק, אריה לויטן

התחיל ב:	19/08/2021, 13:00
מצב	הסתיים
הושלם ב-	19/08/2021, 16:02
הזמן שלקח	3 שעות 2 דקות
מס' אישור הגשה	8199991
ציון	**88.00** מתוך ציון מירבי של 100.00

מידע

קראו בעיון לפני שתתחילו בפתרון הבחינה!

1. המבחן מורכב משלושה חלקים.
2. בחלקים א' ו ב' מופיעות שאלות פתוחות. את התשובות לכל השאלות הפתוחות יש לכתוב בתיבת טקסט .**כל התשובות חייבות להיות מנומקות/מוסברות, גם אם לא כתוב שצריך.** ענו תשובות מלאות בקיצור נמרץ. אין חובה להשתמש בכל השורות המוקצות לצורך התשובות. אין לחרוג מהמקום המוקצה.
3. בחלק ג' (שאלות אמריקאיות) עליכם לבחור בכל פעם בתשובה יחידה מבין התשובות המוצעות.
4. משך הבחינה - שלוש שעות.
5. **ניתן להפנות שאלות לכתובת דוא"ל davidsa.exam@openu.ac.il .**

נא לציין בנושא ההודעה "מבחן 2021ב מועד 87" בגוף ההודעה שם מלא: תז: שאלה מס: סעיף: . כדאי לצרף צילום מסך של השאלה כי לפעמים השאלות מעורבלות.

בהצלחה!

שאלה **1**

הושלם

25.00 נקודות מתוך 25.00

קטע הקוד (פסודוקוד) הבא מיישם פרוטוקול כניסה לקטע קריטי בעזרת פונקצית נעילה LOCK ופונקצית פתיחה UNLOCK שמקבלות את מספר ה- THREAD כפרמטר (המספים ממוחזרים וכל מספר תמיד פחות מ N_THREADS). הקוד מיועד לשימוש בתכניות עם מספר תהליכונים (THREADS).

תוצאת פעולת ההשוואה: (a2,b2) > (a1,b1) היא TRUE אם (b1 < b2) && (a1 < a2)

```
// declaration and initial values of global variables
1. bool willing [N_THREADS] = false;
2. int place [N_THREADS] = 0;
3.
```

```
4. lock(int i ) {
5. willing[ i ] =true;
6. place[ i ] =1+ max(place[0], ..., place[N_THREADS -1]);
7. willing[ i ] =false;
8.  for (int j =0; j < N_THREADS; j++) {
9.    while (willing[ j ] ) { /*do nothing */}
10.    while ((place[ j ] !=0) &&
11.              (( place[ j ] , j ) < ( place[ i ] , i ))) { /*do nothing */ }
12.  }
13. }
14.
15. unlock(int i ) {
16.  place[ i ] =0;
17. }
```

ענו על הסעיפים הבאים **והסבירו/נמקו את כל תשובותיכם:**

א. (7 נק')

הסבירו כיצד הפרוטוקול עובד (איך הוא מבטיח את מניעה הדדית בכניסה לקטע קריטי).

ב. (5 נק')

האם יכול להיות ש THREAD כלשהו יחכה לנצח? אם כן תנו דוגמא לתרחיש, או הסבירו מדוע לא.

ג. (8 נק')

משנים את השורה **6** מ:

([place[i] =1+ max(place[0], ..., place[N_THREADS-1

ל:

place[i] = (1+ max(place[0], ...,place[N_THREADS-1])) %N

כש N_THREADS > int N > 2 .

האם אחרי השינוי מימוש הפרוטוקול עדיין מבטיח את מניעה הדדית? תנו דוגמה נגדית או הוכיחו את הנכונות.

ד. (5 נק')

הוצע לוותר על השורה **9** (למחוק אותה).

while (willing[j]) { */* do nothing */*} .

האם אחרי השינוי מימוש הפרוטוקול עדיין מבטיח את מניעה הדדית? תנו דוגמה נגדית או הוכיחו את הנכונות.

<u>סעיף א</u>

כאשר Thread מסויים רוצה להיכנס לקטע קריטי, הוא דואג א. ששאר הThreadים ידעו שהוא רוצה לקחת "מספר בתור" ע"י שורה 5, המניעה ההדדית מתבצעת ע"י הלולאה בשורה 9 (אם יש מישהו שמנסה לקחת מספר, תחכו עד שהוא יסיים כדי לדעת של מי התור).
ב. הוא בודק את "מקומו בתור". כלומר, שהמקום בתור של שאר הThreadים נמוך משלו וגם שמספר הThread נמוך משלו (העדפה לThreadים עם מספר Thread נמוך).
אם זה לא קורה (יש מישהו לפניו), הוא מחכה עד שהוא יסיים (כלומר עד שאותו Thread קורא לunlock ומאפס את מקומו בתור בשורה 16).

<u>סעיף ב</u>

לא, גם אם הThread הוא האחרון בתור, הוא אכן יחכה לכולם, אך יעשה את הסבב הזה פעם אחת בלבד (בגלל לולאת הFor) [הוא לא יחכה לנצח לThread מסויים בלולאת while מכיוון שהתנאי place[j] =! 0 חייב להיות אמת]. בהנחה שהThreadים באמת קוראים לunlock בסוף השימוש, ברגע שיגיע תורו, הוא ייכנס לקטע הקריטי ולא יחכה לאחרים.

<u>סעיף ג</u>

לא. נקח למשל N = 4 כשיש 6 Threadים. נניח שכל הThreadים לוקחים מספר ע"פ הסדר (הראשון ראשון, השני שני וכו'). הThread הראשון יקבל את מספר 1 ונכנס לקטע הקריטי. כמה שניות לאחר מכן, הThread הרביעי לוקח מספר ובגלל המודולו יקבל גם את מספר 1. בגלל שיש שיוויון במספר התור, הוא גם ייכנס לקטע הקריטי (הוא ידלג על שאר הThreadים כי התור שלו נמוך יותר משלהם), ובכך לא נוצרה מניעה הדדית.

<u>סעיף ד</u>

לא. למשל הThread הרביעי בקטע הקריטי. הThread השלישי קורא ל Lock. ונכנס ללולאת הFor. בזמן הזה, הThread הראשון גם נכנס לקחת תור, אך מכיוון שהורדנו את שורה 9, הThread השלישי לא מחכה שהוא יסיים לקחת את התור, וחושב שהוא (Thread 1) לא מחכה בתור. בזמן הזה הThread הרביעי מסיים, הThread הראשון נכנס לקטע הקריטי, אך גם הThread ה3 נכנס לקטע הקריטי, כי הוא כבר לא מחכה לThread הראשון. בכך לא נוצרה מניעה הדדית. מצב זה יכל להימנע אם היינו משאירים את הלולאה הנתונה.

הערה:

שאלה **2**
הושלם
20.00 נקודות
מתוך 20.00

במערכת עם זיכרון ראשי בגודל 8 GB קיים מעבד עם זיכרון מטמון בגודל 8000 בלוקים ובמערכת רץ תהליך יחיד.
ענו על הסעיפים הבאים **והסבירו/נמקו את כל תשובותיכם:**
א. (4 נקודות)
מהו מספר שורות זיכרון המטמון בהן נדרש לערוך חיפוש בכל פניה לזיכרון ראשי במידה וזיכרון המטמון הוא זיכרון אסוציאטיבי מלא (fully associative) ? נמקו.

ב. (4 נקודות)
מהו מספר שורות זיכרון המטמון בהן נדרש לערוך חיפוש בכל פניה לזיכרון ראשי במקרה של זיכרון מטמון בעל מיפוי ישיר (direct mapping) ? נמקו.

ג. (4 נקודות)
האם התשובה בסעיפים א. ו ב. תשתנה אם במערכת הנ"ל ירוצו 10 תהליכים?
הסבירו את תשובתכם.

סעיפים ד. ו ה. לא קשורים לסעיפים הקודמים
ד. (4 נקודות)

תארו את אלגוריתם ה- next fit לניהול שטחי זיכרון באמצעות רשימה משורשרת.

ה. (4 נקודות)
איזה סוג של ריסוק (fragmentation) מייצר אלגוריתם ה- next fit לניהול שטחי זיכרון באמצעות רשימה משורשרת ומהי השפעתו על מפת השטחים הפנויים/התפוסים?

סעיף א
מכיוון שכל בלוק יכול ללכת לכל שורה בCache, צריך לחפש בכל השורות במקביל (8000). חיפוש זה מתבצע במקביל ע"י חומרה ולכן הוא מהיר יחסית.

סעיף ב
שורה אחת, כל בלוק זיכרון יכול ללכת לבדיוק שורה אחת בCache

סעיף ג
לא, הזיכרון עדיין יעבוד באותה צורה ללא קשר לכמות התהליכים (המעבד לא מקצה מקום בCache לתהליך מסויים אלא בזיכרון עצמו. הגישה לזיכרון היא "דרך" הCache, אבל אין העדפה לתהליך זה או אחר).

סעיף ד
הוא עובד כמו First fit (עוברים על הרשימה עד שמוצאים מקום פנוי), אך במקום להתחיל מהראש כל פעם, הוא שומר את המקום האחרון שבו מצאנו מקום פנוי ובחיפוש הבא הוא מתחיל לחפש משם. כך עד שהוא מגיע לסוף וחוזר לראש.

סעיף ה
external fragmentation, מכיוון שהוא כותב במקום הפנוי הראשון שהוא מוצא (לא מההתחלה, כמתואר בסעיף הקודם), הוא מחלק את הזיכרון לחלקים קטנים. כאשר מגיע תהליך שרוצה חלק גדול בזיכרון, יכול להיות שיהיה מספיק מקום בזיכרון בשבילו, אך מכיוון שהאלגוריתם כותב לפי מקומות שהוא מוצא, האלגוריתם לא ימצא לו מקום, כי יש לו כמה מקומות פנויים אך הם קטנים מידי בשביל התהליך.

הערה:

שאלה **3**

הושלם

8.00 נקודות
מתוך 10.00

במערכת הפעלה מסוימת מערכת קבצים של דיסק קשיח מבוססת I-Nodes .
ענו על הסעיפים הבאים **והסבירו/נמקו את כל תשובותיכם**:
א. (3 נקודות)
האם מבחינת זמן גישה לבלוק יש יתרון להקצאה רציפה של בלוקים של הנתונים לקובץ? בתשובה התייחסו לגישה רציפה ולגישה אקראית.

ב. (2 נקודות)
האם יש יתרון להקצאת בלוקים שמכילים את המצביעים הלא ישרים בסמוך לבלוקים של נתונים של הקובץ?

ג. (3 נקודות)
הוחלט לשנות את שיטת הקצאת בלוקים לקובץ חדש: בזמן יצירת הקובץ יוקצה לו מקום החל מן האמצע של הקטע הפנוי הארוך ביותר שקיים במערכת. מה הן היתרונות של השיטה הזאת?

ד. (2 נקודות)
כשמדובר באותה שיטה מסעיף ג', באיזו צורה (רשימה משורשרת ממוינת או מפת סיביות) עדיף לשמור את הנתונים לגבי מקום פנוי במערכת הקבצים (ניתן לשנות את השיטה המקורית המקובלת במערכת קבצים מבוססת I-Nodes) ?

סעיף א
כן, כי בהקצאה רציפה צריך רק קריאה אחת כדי לקבל את כל הנתונים, לעומת זאת, בגישה אקראית צריך כמה קריאות (בהתאם לחלוקה).

סעיף ב
כן, כך שתנועת הזרוע של הדיסק צריכה ללכת פחות.

סעיף ג
יש לו את האפשרות לגדול ברציפות בלי חסימה של קבצים אחרים. לעומת זאת אם היינו מקצים מההתחלה, הקובץ שבא לאחריו ייתן לו חסם עליון מסוים מבחינת המקום הרציף.

סעיף ד

רשימה משורשרת, כך אפשר בקלות יותר למצוא את המקום הגדול ולעבור ממקום למקום.

הערה:
א. בכל מקרה צריך יהיה לבצע כמה קריאות. כל בלוק נקרא בנפרד

א. בכל מקרה צריך יהיה לקרא את הכניסה הבאה הרי
הוגדר שבשיטת I-NODES. השאלה היא האם בכל זאת יש
עדיפות להקצאה רציפה כשאפשר.
-2

שאלה **4**
הושלם
5.00 נקודות
מתוך 5.00

במערכת XV6 שתומכת בקונטיינרים (אבל לא בקונטיינרים מקוננים nested) הפעילו 2 רצפים של הפקודות, **כל רצף אחרי הרצה נקייה של המערכת:**

רצף 1	רצף 2
pouch start c1	pouch start c2
pouch start c2	pouch connect c2
pouch connect c2	pouch start c1
mkdir d2	**הודעת שגיאה//**
pouch disconnect	mkdir d2
pouch connect c1	pouch disconnect
mkdir d1	
pouch disconnect	

במקרה של רצף 1 כל הפקודות הצליחו, ובמקרה של רצף 2, היתה הודעת שגיאה(איפה שכתוב).
מה הייתה הודעת השגיאה **הצפויה?** הסבירו למה.

nesting containers is not supported.

מכיוון שהמערכת לא תומכת בקונטיינרים מקוננים (כנתון), אין את האפשרות לפתוח עוד קונטיינר בו בזמן שאנחנו נמצאים בקונטיינר C2

הערה:

שאלה **5**
הושלם
5.00 נקודות
מתוך 5.00

במערכת XV6 שתומכת בקונטיינרים (אבל לא בקונטיינרים מקוננים nested) הופעל רצף הבא של הפקודות **אחרי הרצה נקייה של המערכת:**

פקודות
$ pouch start c1
Pouch: c1 starting
$ cat cgroup/cgroup.procs
1
2
8
6
$ pouch connect c1
tty0 connected
$ cat cgroup/cgroup.procs
//nothing printed
$ pouch disconnect
Console connected
cat cgroup/cgroup.procs
1
2
14
6
$

הסבירו את הפלט, **התייחסו רק לנקודות הבאות:**
א. למה הפלט **היה ריק** איפה שכתוב //nothing printed
ב. מספר 8 בפלט הראשון הוא PID של הפקודה cat , בפלט השני מספר 14 שהוא PID של הפקודה cat השלישית. למה הקפיצה הייתה כה גדולה, הרי המערכת אמורה להקצות PID –ים ברצף לכל תהליך חדש שנוצר.

<u>סעיף א</u>
מכיוון שזה קונטיינר חדש, אין בו שום processים, ולכן לא היה שום מידע להדפיס.

<u>סעיף ב</u>
הספירה של התהליכים כוללת את מה שנכתב בקונטיינר, ולכן זה קפץ כך.

הערה:

שאלה **6**
הושלם
0.00 נקודות
מתוך 5.00

במערכת XV6 שתומכת בקונטיינרים (אבל לא בקונטיינרים מקוננים nested) הופעל רצף הבא של הפקודות **אחרי הרצה נקייה של המערכת:**

פקודות
$ pouch start c1
Pouch: c1 starting
$ pouch connect c1
tty0 connected
$ mkdir mntdir
$ mount mntdir -t cgroup
cgroup filesystem already mounted
$

מדוע פקודת mount נכשלה והודעה היתה " already mounted " , הרי המשתמש מעולם **לא הפעיל ישירות** את הפקודה הזאת? על ידי מי ולמה פקודת mount כבר הופעלה?
הסבירו את תשובתכם.

ברגע שפותחים את המערכת, אנחנו נמצאים בGlobal Mount Namespace, על מנת להתחיל namespace חדש, צריך להשתמש קודם ב unshare

הערה:
אם כך, מדוע נכשלה הקריאה?
חסר התשובה

שאלה **7**
הושלם
5.00 נקודות
מתוך 5.00

כידוע קיימים 2 סוגים של HYPERVISOR –ים **בצורת HOSTED** (מבוססים על מערכת ההפעלה המארחת): סוג אחד תומך רק ב פרה-וירטואליזציה והסוג השני תומך בוירטואליזציה מלאה וממומש בשיטה Trap-And-Simulate . האם בשני הסוגים חייב להיות מודול (MODULE) של HYPERVISOR שיפעל כחלק מגרעין מערכת ההפעלה המארחת (HOST OS) וייטען ביחד עם ה KERNEL בזמן עליית המערכת?
הסבירו את תשובתכם.

כן. בוירטואליזציה מלאה חייב שיהיה מודול כזה, שכן צריך להחליף את שורות הקוד הרגישות. בפרה-וירטואליזציה, החומרה לא מכירה hypercalls (מה שמחליף את החלק הרגיש) ולכן חייב שיהיה מודול שיתרגם אותם.

הערה:

שאלה **8**
הושלם
5.00 נקודות
מתוך 5.00

בניסיון למנוע את מצב הקיפאון DEADLOCK במערכת כלשהי נקבע הפרוטוקול הבא:
א. לכל משאבי המערכת יש מספרים סידוריים.
ב. כל תהליך יכול לבקש הקצאת משאבי המערכת רק בסדר עולה של המספרים ולשחרר אותם אחרי גמר השימוש רק בסדר יורד.
האם הפרוטוקול הזה אכן מונע את אפשרות הקיפאון?
הוכיחו שכן או הפריכו (ע"י דוגמא נגדית).

נניח בשלילה כי האלגוריתם אינו מונע קיפאון. יהי תהליך a,b הדורשים את משאבים 3,4 (לפי הסדר). תהליך a רוצה לקבל את המשאבים, וע"פ האלגוריתם יתחיל מלמטה ויגיע ל3. לפי ההנחה בשלב הזה תהליך b לקח כבר את משאב 4 ולא את 3 (כדי שהאלגוריתם לא ימנע קיפאון) והגענו לקיפאון. אך זה עומד בסתירה לאלגוריתם, שכן כדי שתהליך b ייקח את 4 הוא צריך לקחת קודם את 3. וכן הוא חייב לשחרר בסדר יורד. הגענו לסתירה ובכך הוכחנו שהאלגוריתם מונע קיפאון.

הערה:

שאלה **9**
תקין
5.00 נקודות
מתוך 5.00

איזו פעולה מן הפעולות הבאות אפשר לבצע אך ורק במצב ראשוני (kernel mode) במערכת ההפעלה Linux?

יש לבחור תשובה אחת:
- את כל שלושת הפעולות המפורטות בשאלה אפשר לבצע אך ורק במצב ראשוני
- החלפת תהליכונים (thread switch) כאשר מדובר בספריית תהליכונים ברמת המשתמש
- חסימת פסיקות החומרה (disabling hardware interrupts) ✔
- השמת ערך במשתנה גלובאלי

תשובתך נכונה.

שאלה **10**

בחרו את הפעולה היקרה ביותר במונחים של מעברי בלוקים מן הדיסק (disk block transfers) במערכת קבצים בצורת I-Node בהנחה

שלא קיימים נתונים רלוונטיים בזיכרון המטמון (buffer cache):

יש לבחור תשובה אחת:

- קריאת תו אחד באמצעות getc מהתחלה של הקובץ ✖
- קריאת 2 בלוקים באמצעות פקודת read מן ההתחלה של קובץ קטן שתופס 10 בלוקים בסה"כ
- קריאת בלוק אחד באמצעות read מן הסוף של הקובץ בגודל המירבי האפשרי במערכת
- לא ניתן לקבוע בוודאות מה התשובה הנכונה וזה תלוי במידת פיזור הבלוקים של הקובץ

תשובתך אינה נכונה.

שאלה **11**

תקין

5.00 נקודות
מתוך 5.00

במערכת עם ניהול זיכרון מדופדף נהוגה מדיניות של prepaging – הבאת מספר כלשהו של דפים השייכים לתהליך עובר מהדיסק לזיכרון. בחרו את הטענה הנכונה:

יש לבחור תשובה אחת:

- הבאת קבוצת דפים אקראיים היא פעולה שבד"כ תוריד את כמות פסיקות הדפים במערכת
- מדיניות זו יכולה להקטין את מספר פסיקות דף אם הדפים בקבוצה נבחרו ע"י אלגוריתם מתאים ✔
- מדיניות זו עומדת בסתירה לעקרון הלוקליות בזמן ובמקום
- אין שום טעם להביא דפים מראש שכן הדבר כרוך בבזבוז זמן על הפינוי בשלב מאוחר יותר

תשובתך נכונה.

שאלה **12**

תקין

5.00 נקודות
מתוך 5.00

מערכת הקבצים של מערכת ההפעלה מסוימת משתמשת בשיטת ה- I-node.
גודל הבלוק במערכת הקבצים הוא 1 KB.
כתובת הבלוק היא 8 בתים (bytes).
10 שדות של ה-I-node יכולים להחזיק ישירות כתובות הבלוקים בדיסק.
שדה נוסף אחד נועד להחזיק כתובת של ה-single indirect block.
עוד שדה נוסף אחד נועד להחזיק כתובת של ה-double indirect block.
ועוד שדה נוסף אחד נועד להחזיק כתובת של ה-triple indirect block.
מהו הגודל המקסימאלי של קובץ במערכת הקבצים הזו?
המספרים מעוגלים לחצאי ג'יגה ביתים.
למשל 1.567 ג'יגה בית יעוגל ל 1.5 ג'יגה ו 1.82 ג'גה יעוגל ל 2 ג'יגה.
יש לבחור את התשובה הקרובה ביותר. בית = byte .

יש לבחור תשובה אחת:

- 2 ג'יגה בית ✔
- 4 ג'יגה בית
- 1.5 ג'יגה בית
- 2.5 ג'יגה בית

תשובתך נכונה.